
AI3002-25Fall Final Project: 强化学习实验报告

郝思粤 (PB23151779)

姓名 B (学号)

姓名 C (学号)

中国科学技术大学

{hsy20050214,emailB,emailC}@mail.ustc.edu.cn

Abstract

1 Torch 配置与环境说明

2 Warm-up: 经典强化学习

2.1 Monte Carlo Learning: Blackjack

2.1.1 First-visit MC: 核心思想、更新对象与策略改进 (必做)

2.1.2 实验设置与结果展示 (必做)

2.1.3 收敛特点与方差来源讨论 (必做)

2.1.4 Every-visit MC: 对比与讨论 (选做)

2.2 Temporal-Difference Learning: CliffWalking

2.2.1 SARSA 与 Q-learning: 更新差异 (必做)

2.2.2 训练曲线对比与现象解释 (必做)

2.2.3 Double Q-learning: 对比与动机 (选做)

3 实验原理 (主任务)

3.1 二元零和马尔可夫博弈 (Two-Player Zero-Sum Markov Game)

在本实验中, 我们将问题建模为二元零和马尔可夫博弈。具体而言, 该过程可以由一个元组 $(S, A_1, A_2, P, R, \gamma)$ 描述:

- **马尔可夫性**: 环境的下一状态 s_{t+1} 仅取决于当前状态 s_t 以及双方采取的联合动作 $(a_{1,t}, a_{2,t})$, 与历史状态无关。
- **二元零和**: 系统中存在两个智能体, 且双方的利益完全对立。对于任意状态和动作, 满足 $R_1 = -R_2$, 即一方的收益等同于另一方的损失, 也就是说, 两个人的收益“和为零”。这意味着博弈不存在双赢, 而是“你死我活”, 双方通过极大极小策略进行对抗。

与单智能体 MDP 的差异: 在传统的单智能体强化学习 (MDP) 中, 环境的状态转移概率通常是固定的。而在本实验的博弈环境中, 对于智能体 A 而言, 环境包含了智能体 B。由于智能体 B 也在不断学习和调整策略, 导致环境具有**非平稳性 (Non-stationarity)**。这使得简单的 MDP 算法难以直接收敛, 因为最优策略是随着对手的变化而动态变化的 (寻找纳什均衡点)。

3.2 Naive Self-Play (朴素自博弈)

由于人类专家比较昂贵, 并且实验并没有提供黑盒的强大 opponent, 因此本实验采用 **Naive Self-Play** 机制进行训练。

- **对手选择机制**：在训练过程中，智能体不仅要与当前的“自己”对抗，更多时候是与历史版本的“自己”（Past Checkpoints）进行对弈。我们在训练每隔固定轮次（Epoch）后保存模型快照，并在新一轮训练中随机采样旧模型作为对手。
- **优势与风险**：
 1. **课程学习效应**：随着训练进行，作为对手的“自己”越来越强，相当于构建了一个难度自动升级的敌人，迫使当前 agent 不断进化。
 2. **潜在缺陷**：虽然五子棋总体上具有传递性（即高水平模型通常优于低水平模型），但 Naive Self-Play 容易导致模型“遗忘”早期策略。具体表现为：当模型专注于应对高水平对手的复杂战术时，可能会丢失对简单或非典型（随机）策略的防御能力。为了缓解这一问题，我们在对手池中保留了部分早期模型和随机策略模型，以保证策略的鲁棒性。

3.3 Actor-Critic 架构

为了解决博弈中的策略优化问题，我们采用了 Actor-Critic 架构，该架构包含两个核心神经网络：

- **Actor（策略网络）** $\pi(a|s; \theta)$ ：负责“执行”。输入当前状态 s ，输出动作的概率分布。其目标是最大化 Critic 评估的期望回报。
- **Critic（价值网络）** $V(s; w)$ ：负责“打分”。输入当前状态 s （或状态动作对），输出对当前局面的价值预估。其目标是最小化预测价值与真实回报之间的误差。

原理与协作机制：Actor-Critic 结合了基于价值（Value-based）和基于策略（Policy-based）的方法。在训练过程中，Critic 通过时序差分误差来更新参数，充当这一步动作的“裁判”；Actor 则根据 Critic 计算出的**优势函数（Advantage Function）**来更新策略梯度。引入 Critic 的主要目的是为了降低梯度的方差，相比于单纯的 Monte-Carlo 采样（需要等到回合结束），Critic 提供的低方差基线使得训练更加稳定。

损失函数分析：

- **Actor Loss**：衡量策略改进的方向，旨在提高高价值动作的概率。
- **Critic Loss**：通常为均方误差（MSE），衡量价值估计的准确度。
- **Entropy（熵）**：为了防止模型过早陷入局部最优，我们在 Loss 中引入熵正则项，鼓励 Actor 保持一定的探索性（Exploration）。

4 模型设计

4.1 __init__ 与 forward 方法

我们先介绍代码中需要补全的三个类 Actor、Critic 和 GobangModel 的设计动机与实现细节。

1. Actor 类 设计动机与实现： Actor 是用来实现“下棋”这个动作，其将复杂的棋盘局面映射为具体的落子概率。在实现过程中需要注意的点是，如果不加以修正，模型可能会尝试在已有棋子的位置重复落子。为了避免模型在训练早期浪费大量算力去试错，我们引入了棋盘坐标的“合法性约束”。即在输出层引入 `legal_mask`，利用矩阵掩码操作将所有非法落子点的 Logits 强制推向负无穷，因为 softmax 的公式是 $P(a|s) = \frac{\exp(\text{logits}(a|s)/\text{temp})}{\sum_{a'} \exp(\text{logits}(a'|s)/\text{temp})}$ ，所以这种令概率为负无穷的操作的作用是，使得数学上让 Softmax 后的非法概率严格归零，也就是说，模型不会尝试在“已经有棋”的地方下棋。同时，我们在 `forward` 中加入了一个温度参数 `temp`。在训练初期，通过调高温度让概率分布更平坦，从而鼓励模型探索一下更多的落子可能性，防止其过早陷入某些局部最优的套路中。

Listing 1: Actor 类的核心实现

```

1 def __init__(self, board_size, d_model=128, nhead=8):
2     super().__init__()
3     self.trunk = BoardTransformer(board_size, d_model=d_model, nhead=
        nhead)
4     self.logits_head = nn.Linear(d_model, 1)
5
6 def forward(self, x):
7     feat = self.trunk(planes)
8     logits = self.logits_head(feat).squeeze(-1)
9     masked_logits = logits.masked_fill(~legal_mask, -1e9)
10    probs = torch.softmax(masked_logits / self.temp, dim=-1)
11    return probs

```

2. Critic 类 设计动机与实现： 在 Actor-Critic 架构中，Critic 能够知道当前行动的优劣，负责对当前落子进行“打分”。我们最初考虑让 Critic 直接输出一个标量，但实验发现这样会导致信号过于模糊，网络难以捕捉细微的落子差异。因此，我采用了“全图价值映射”的设计思路：先让网络计算出棋盘上每一个位置如果被占据后可能带来的潜在价值（Value Map），然后再根据 Agent 实际执行的动作，利用 `gather` 函数精准地从中提取出对应点的 Q 值。这种实现方式的优势在于，它强迫神经网络在底层特征提取时就必须对棋盘的每一个区域进行价值建模，使得特征表示更加丰富。在反向传播时，这种“先全图评分再特定提取”的逻辑能为骨干网络提供更密集的梯度信号，极大地缓解了奖励信号稀疏导致的评分器训练缓慢问题。

Listing 2: Critic 类的核心实现

```

1 def forward(self, x, action_xy):
2     feat = self.trunk(planes)
3     q_map = self.q_head(feat).squeeze(-1)
4     idx = action_xy[:, 0] * self.board_size + action_xy[:, 1]
5     return q_map.gather(1, idx.unsqueeze(1)).squeeze(1)

```

3. GobangModel 类 设计动机：作为集成模块，它不仅负责封装 Actor 和 Critic，还在 `optimize` 方法中实现了优势函数 (Advantage) 的计算，通过 $A(s, a) = Q(s, a) - V(s)$ 的思想减少策略梯度更新时的方差。

4.2 自主设计

我们发现仅仅对 `submissionion` 进行基础补全的模型性能一般，为了使得模型更聪明的下棋，我们引入了一些进阶设计，接下来我们将分别介绍改进内容。

温馨提示助教哥哥：由于这部分的改进是我们在模型优化中一步步探索出来的，因此这里仅做总结介绍，具体实现动机、性能提升的实验对比佐证我们重点放在了“10. 模型优化：超参数与结构改进记录”一节中，为什么引入如下小巧思请见下回分解~

1. Transformer 骨干网络 传统的 CNN 感受野有限，难以捕捉五子棋中横跨整个棋盘的连线威胁。合格的棋手并不能只盯着棋盘的某个局部，因此我们使用了 `BoardTransformer` 替代了 `ResNet`。利用 **Self-Attention 自注意力机制**，模型在下子前可以同时观测到棋盘上所有已有的棋子，从而具备更好的全局大局观。

2. 启发式引导 (Heuristic Shaping) 强化学习初期完全靠随机探索，效率极低。我想给模型注入一些“棋手的常识”。**关键代码：**

```
1 def _eval_shaping_logits(self, board, legal_mask):
2     # 检测必胜手（己方连四）或必救手（对方连四）
3     # 为这些关键格子的 Logits 增加额外的 Bias
4     for idx in cand_idx:
5         own_len = self.check_max_line(board, x, y, self_color)
6         opp_len = self.check_max_line(board, x, y, opp_color)
7         if own_len >= 5: score += 10000 # 必胜
8         if opp_len >= 5: score += 9000 # 必救
9     return torch.tensor(out)
```

3. 重写 train 函数与视角转换逻辑 我们认为原始 `utils` 中的训练脚本不够灵活。因此便在 `submissionion.py` 中实现了新的 `train` 函数，其有如下两个特点：

- 1. Identity Transform (颜色对称)：**在原始训练过程中，仅利用黑棋的结果进行反向传播，因此我们便调整了训练过程，在 Agent 执白棋时，通过棋盘颜色翻转，让模型“以为”自己在执黑。这样一套神经网络就能通吃黑白双方，数据利用率提升一倍。
- 2. 两阶段调度：**在训练过程中，前 400 轮将敌人先设置为“随机对手”，随后再将敌人设置为“历史对手池 (Opponent Pool)”中随机“捞”出来的一个，经实验验证，模型在此种训练方式下更加稳定（见“10. 模型优化”一节）。

5 约束策略分析：置 0 限制和归一限制

在五子棋 AI 的决策过程中，模型输出的策略必须严格遵守博弈规则，即“受限策略 (Constrained Policy)”。本实验的 `forward` 实现逻辑与 Gu & Sung 提出的 *Enhanced Reinforcement Learning Method Combining One-Hot Encoding-Based Vectors for CNN-Based Alternative High-Level Decisions* 中“合法性约束 + 次优落点”思路一致 [Gu and Sung, 2021]，有效解决了非法落子限制与概率合法性问题。

5.1 置 0 限制：非法落子的硬性约束

理论逻辑： Gu & Sung 指出，传统 CNN 在占子位置上会输出无效动作，需通过显式约束排除不可用落点，才能选择“次优位置”[Gu and Sung, 2021]。

实现： 我们通过 **Logits Masking (掩码填充)** 技术实现了这一限制。

- **状态感知：** 预处理阶段通过 `_board_to_planes` 函数提取出空位掩码 `legal_mask`。
- **分数值截断：** 在进入 `Softmax` 激活函数前，执行 `logits.masked_fill(~legal_mask, -1e9)`。这一步将所有非空格点的原始得分 (Logits) 设为极小值 (-10^9)。
- **置零效果：** 由于 `Softmax` 涉及指数运算 e^x ，当 $x = -10^9$ 时，其输出值无限趋近于 0，从而在逻辑上实现了非法落子点的“置 0 限制”。

5.2 归一限制：合法概率分布的构建

理论逻辑： 论文利用 One-Hot 编码组合让概率分布自动在合法位置间重分配 [Gu and Sung, 2021]，当首选被屏蔽时概率应流向其他可落子点。

实现： 为了确保概率之和严格等于 1，代码采用了 **两步重归一化 (Renormalization)** 机制：

- **全局映射：** 初始 `Softmax` 计算出全图 144 个格子的基础概率分布。
- **概率重分配：** 使用代码 `probs = probs / (probs.sum(dim=1, keepdim=True) + 1e-8)`。
- **数学原理：** 当非法位置的概率被置为 0 后，剩余合法位置的概率之和 $\sum P_{legal}$ 会小于 1。通过除以当前概率的总和，代码将剩余的概率按比例放大，确保了最终输出是一个数学上严谨的概率分布。

6 Bug 修复：optimize 函数中的三个问题

6.1 Bug 1: TD 目标值梯度更新错误

- **Bug:** 在计算目标值 $y = r + \gamma Q(s')$ 时，没有对下一状态的 Q 值执行脱离计算图的操作，导致反向传播时梯度会错误地流向“未来的预估值”，造成训练震荡且不符合 TD 学习算法的理论逻辑。
- **Solution:** 对 `next_qs` 调用 `.detach()`，确保目标值在计算损失时被视为常数，从而只更新当前状态的 Q 网络。

6.2 Bug 2: Actor 权重更新缺失

- **Bug:** 代码中虽然调用了 `actor_loss.backward()` 计算梯度，但漏掉了最关键的步进操作。这导致策略网络的参数在内存中虽然有梯度，但从未真正执行过更新，模型无法进化。
- **Solution:** 我们在反向传播之后补上了 `self.actor.optimizer.step()`，使优化器根据梯度修正 Actor 的神经网络权重。

6.3 Bug 3: Critic 权重更新缺失

- **Bug:** 与 Actor 类似，Critic 网络虽然计算了 MSE 损失并回传了梯度，但由于缺少步进指令，评价器的参数始终保持初始状态，无法根据奖励信号修正其对局势的评分（ Q 值）。
- **Solution:** 在 Critic 的损失函数回传后增加 `self.critic.optimizer.step()`，确保评分标准能随训练不断优化。

7 其他问题记录（工程与调试）

8 疑难点思考

9 曲线分析：Loss 与 Entropy

9.1 指标定义与期望趋势

9.2 结合 `loss_tracker.png` 的阶段性感读

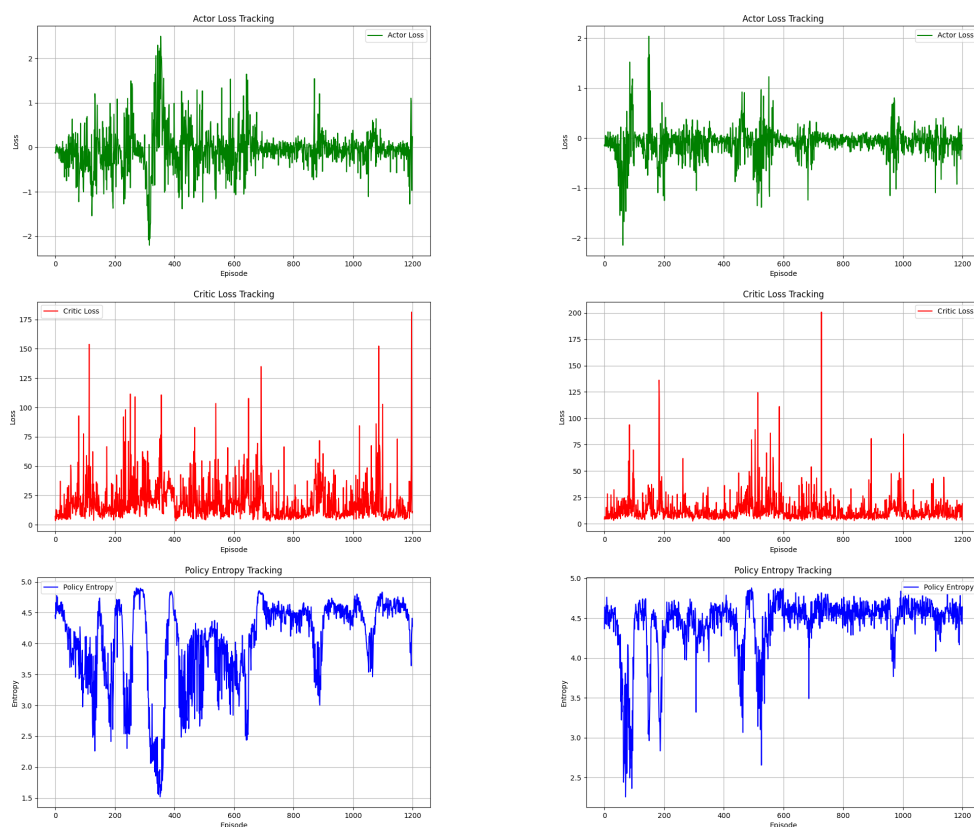
10 （选做）模型优化：超参数与结构改进记录

10.1 训练策略优化

在助教提供的 `utils` 文件中，模型训练过程中的敌人始终是自己，这会导致模型容易收敛较慢或者陷入局部最优。因此，我们在 `submission.py` 中新增了优化过的 `train` 函数，其有两个显著变化：

1. **两阶段训练策略**：在训练的前 400 个 Epoch 中，敌人始终为“随机对手”，以帮助模型快速学习基本规则和简单策略。随后在后续 Epoch 中，敌人切换为“对手池（Opponent Pool）”中的历史版本，以促进模型的持续进化。
2. **视角转换（颜色对称）**：在模型执白棋时，通过棋盘颜色翻转，让模型“以为”自己在执黑。这样一套神经网络就能通吃黑白双方，数据利用率提升一倍。

并且，经过实验验证，在胜率并没有下降的情况下，优化过训练过程的模型收敛速度更快，稳定性更好。具体胜率曲线对比如图 1 所示：



(a) 阶段一：随机对手

(b) 阶段二：随机噪声 + 对手池

图 1：不同训练策略下模型胜率对比。

左图为 `utils` 中定义的训练过程的收敛曲线，右图为使用优化过的 `train` 函数进行训练得到的曲线，两图对应模型的所有参数一致，训练轮数为 `episodes=1200`。可以看出，右图中的每一条曲线均在后半部分更为平滑，震幅不那么剧烈，说明右图对应的模型收敛效果更好。

10.2 结构与超参数优化

本实验采取了“渐进式升级”的策略，通过架构创新与系统的网格搜索（Grid Search）不断提升模型强度。优化过程分为三个主要阶段：基础基准对齐、特征工程增强以及全局视野引入。

1. 基础 CNN 基准构建与内部赛马 最初，我们实现了一个基础的卷积神经网络（CNN）架构作为起点。该版本输入为简单的三维张量（己方棋子、对方棋子、空位）。

- **优化动机：**在复杂的强化学习实验中，首先需要建立一个可靠的基线（Baseline）。我们需要在最基础的结构上找到一组稳健的超参数，以确保后续的架构改进具有可比性。
- **实验过程：**通过内部“赛马”机制，我们对学习率（ 2×10^{-4} 到 5×10^{-4} ）和网络深度（Residual Blocks）进行了初步筛选。
- **阶段结论：**最终选出一组在训练稳定性与收敛速度上表现最优的模型，作为后续实验的初级敌人基准（Fixed Baseline）。

2. 领域知识引入与 CNN+ 增强版优化 在基础 CNN 之上，我们引入了博弈领域的先验知识进行特征工程，将模型升级为 CNN_plus。

- **优化动机：**纯图像化的棋盘表示缺乏对五子棋核心博弈逻辑的直接引导。在实际对局中，“最后一次落子位置”（Last Move）往往决定了当前的攻防中心，而“气”（Liberties）的多少直接反映了棋子的灵活性和被包围的风险。
- **架构改进：**输入通道由 3 通道扩展为 5 通道。新增通道 4 记录对方最后一步坐标（Last Move Plane），通道 5 通过卷积算子实时计算并归一化棋盘各点的“气”状态（Liberties Plane）。
- **网格化搜索：**利用 `grid_search_cnn.py`，我们将增强版 CNN 与初级基准进行大规模对弈。搜索维度涵盖了通道数、残差块数量以及评估阶段的温度系数（Temperature）。
- **实验结果：**通过“赛马”筛选出胜率前 10 名的参数组合，形成了更为强大的“精英对手池”，作为下一阶段架构挑战的目标。

3. 全局视野引入：Attention 架构创新 最后，我们尝试跳出卷积核局部视野的限制，实现了基于 Transformer 结构的 `submission_attention` 模型。

- **优化动机：**CNN 在处理长距离依赖（如跨越半个棋盘的斜向五连）时，需要极深的网络才能获得足够的感受野。而 Attention（自注意力机制）天然具有全局视野，能够同时关注棋盘上任意两个落子点的逻辑关联。

- **网格化搜索与对齐**: 使用 `grid_search_attention.py`, 让 Attention 模型作为黑棋挑战前一阶段筛选出的 10 组 Elite CNN。我们针对嵌入维度 (`d_model`)、注意力头数 (`nhead`) 和 Dropout 进行了精细调节。
- **最终结论**: 实验证明, 在同等参数规模下, Attention 模型在应对复杂攻防博弈时展现出比 CNN+ 更强的泛化能力, 尤其是在应对非典型开局时表现更为稳健。

11 (选做) 思考题: Loss 最优点是否必然达到纳什均衡?

我们认为**不必然达到**。在本实验(五子棋自博弈)里, 把训练的 Loss 降到很低, 通常只能说明: 模型在当前训练设置和当前对手分布下变强了。但这并不等价于“已经到达纳什均衡”。直观地说, 纳什均衡更像是一种“谁单方面改策略都占不到便宜”的稳定状态; 而 Loss 下降只是“这段时间内我学得会更会下棋了”, 两者并不是一回事。

11.1 原因

1. 对手在变, 训练目标也在变 在单智能体任务里(比如走迷宫), 环境规则基本固定, 训练过程中 Loss 的下降比较容易对应“越来越接近最优策略”。但自博弈里, 对手本身也是模型(或者历史版本的自己), 它的水平会随着训练变化。这样就会出现一种情况: **我今天学到的“最优应对”, 可能只是对昨天那个对手最有效**; 等对手更新后, 这个“最优”就不一定还是最优了。所以 Loss 达到某个低点, 很多时候只是“对某个阶段的对手”学得很好, 并不能保证对所有可能的对手都稳。

2. 可能出现“互相克制”的循环 零和对抗里常见一种现象: 策略之间不一定有严格的“强者恒强”。更直观的例子是“剪刀石头布”: 石头克剪刀、剪刀克布、布又克石头。如果训练过程总是追着当前对手的弱点去改进, 就可能出现来回循环: 我克你一下, 你换个套路又克回来。这说明: **即使 Loss 下降得很顺, 也不代表最终会停在一个真正稳定的位置**(也就是纳什均衡那种“谁改都没便宜占”的状态)。

3. 只对“训练中遇到的对局”表现好, 不代表全局都稳 我们的训练数据来自 rollouts (对局采样)。模型优化的其实是“在这些采样到的局面上赢得更多”。但真实的博弈空间非常大, 训练中不可能覆盖到所有关键局面。于是可能发生: 模型对训练时常见的套路很强, 但遇到某些少见/针对性的走法就暴露漏洞。换句话说, Loss 很低可能是“在常见样本上做得很好”, 并不等于“对任何对手都不会被针对”。

4. 神经网络的近似与训练噪声会带来偏差 五子棋的策略空间很复杂, 而我们的模型容量、训练轮数、探索强度都有限。再加上 Actor-Critic 里价值评估(Critic)的误差、采样的随机性等因素, 都会导致训练信号并不完全可靠。结果就是: **Loss 看起来很漂亮, 但策略并不一定真的达到了理论上的稳定最优**。

总的来说, Loss 更像是训练过程中的“方向指示器”, 它告诉我们相对之前有没有进步; 而纳什均衡更像是对抗博弈里的“稳定终点”。在本实验里, 我们引入对手池 (Opponent

Pool) 和阶段性自博弈，本质上就是让模型不要只针对单一对手过拟合，而是尽量在更多不同风格的对手面前都能站得住脚。虽然这能让策略更稳健、更接近“稳定状态”，但仅凭 Loss 最优并不能保证一定到达纳什均衡。

12 课程反馈

本小组大约花了四天的时间去完成实验，在主实验中花了较长的时间去优化模型、使用网格化搜索去提升模型的强度。在实验过程中，我们遇到了不少困难与，比如 Actor-Critic 框架的理解、Transformer 架构的实现、模型优化的方向等，但通过查阅资料和高频率的腾讯会议小组讨论，最终都一一克服了。总的来说，从分工到实验结束，大约花了四天的时间去完成实验报告的撰写与代码实现，收获颇丰。

References

X. Gu and Y. Sung. Enhanced reinforcement learning method combining one-hot encoding-based vectors for cnn-based alternative high-level decisions. *arXiv preprint arXiv:xxxx.xxxxx*, 2021.

A 附录 A：小组分工（建议放表格）

B 附录 B：模型优化过程关键代码

B.1 特征工程增强 (CNN+) 实现

在 `submission_cnn_plus.py` 中，我们将领域知识转化为神经网络可理解的特征平面。以下为“气”平面的实时计算逻辑：

Listing 3: 气 (Liberties) 平面的卷积实现

```
1 def _liberties_plane(board_1ch: torch.Tensor) -> torch.Tensor:
2     """计算各位置相邻空位 (4-连通) 的数量并归一化"""
3     empty = (board_1ch == 0).to(torch.float32)
4     # 定义 4-连通卷积核
5     kernel = torch.tensor([[0, 1, 0], [1, 0, 1], [0, 1, 0]], device=
6         device).view(1, 1, 3, 3)
7     # 利用 conv2d 快速计算全图邻域状态
8     neigh = F.conv2d(empty, kernel, padding=1)
9     return torch.clamp(neigh, 0.0, 4.0) / 4.0 # 归一化到 [0, 1]
```

B.2 网格搜索对弈评估逻辑

在 `grid_search_attention.py` 中，我们通过自动化对弈实现不同架构间的“性能对齐”评估：

Listing 4: Attention vs Top-CNN 赛马评估核心

```

1 def evaluate_winrate(model_black: AttnModel, model_white: CnnModel,
2   episodes: int) -> float:
3     board = Gobang(board_size=12, bound=5, training=False)
4     board.model = model_black # 待评估的 Attention 架构
5     board.opponent = model_white # 之前阶段筛选出的 Elite CNN 基准
6     black_wins = 0
7     for _ in range(episodes):
8         board.restart()
9         while True:
10             # 禁用随机探索, 测试模型真实强度
11             color, end_up = board.update_board(learning=False,
12               random_response=False)
13             if end_up:
14                 if color == 1: black_wins += 1 # 统计黑棋胜场
15                 break
16     return black_wins / episodes

```

B.3 Transformer 棋盘 Token 化与位置编码

为了将 2D 棋盘适配 Transformer 架构, 我们在 `submission_attention.py` 中实现了特征序列化与位置信息注入:

Listing 5: Transformer 棋盘序列化表示

```

1 class BoardTransformer(nn.Module):
2     def __init__(self, board_size, d_model, ...):
3         super().__init__()
4         self.tokens = board_size * board_size
5         self.in_proj = nn.Linear(3, d_model) # 特征投影
6         self.pos = nn.Parameter(torch.zeros(self.tokens, d_model)) # 学习
7           2D 位置权重
8
9     def forward(self, planes: torch.Tensor):
10         # planes: (B, 3, N, N) -> 转化为序列 (B, 144, 3)
11         b, c, n, _ = planes.shape
12         x = planes.permute(0, 2, 3, 1).reshape(b, n * n, c)
13         # 注入全局空间位置信息后进入 Encoder
14         x = self.in_proj(x) + self.pos[None, :, :]
15         return self.encoder(x)

```

C 附录 C: 补充实验曲线与更多评测结果 (可选)